❑ 개념 확인

(1) 스레드를 생성하는 방법 두 가지는 무엇인가요?

===== 풀이 ====

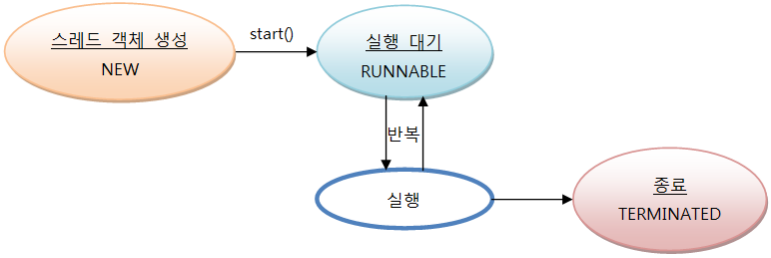
:(1) Thread 클래스를 상속받는 방법

:(2) Runnable 인터페이스를 구현하는 방법

(2) 스레드의 일반적인 상태에 대하여 설명하세요

===== 풀이 ====

- 스레드 객체를 생성하고 start() 메소드를 호출하면 바로 실행되는 것이 아니라 실행 대기 상태가 됨
- 실행 상태의 스레드는 run() 메소드를 모두 실행하기 전 다시 실행 대기 상태로 돌아갈 수 있음
- 실행 대기 상태에 있던 다른 스레드가 선택되어 그 스레드가 실행 상태가 되기도 함
- 실행 상태에서 run() 메소드의 내용이 모두 실행되면 스레드의 실행이 멈추고 종료 상태가 됨



(3) 다음의 메소드는 어떠한 경우에 사용되나요?

① sleep()
② yield()
③ start()
④ join()
⑤ interrupt()

===== 풀이 ====

① sleep() : 주어진 시간 동안 스레드 일시 정지
② yield() : 다른 스레드에게 실행을 양보
③ start() : 스레드를 시작, 스레드가 실행 대기 상태가 됨
④ join() : 다른 스레드의 종료를 기다림
⑤ interrupt() : 현재 스레드를 중단

(4) 스레드 상태 제어를 하는 메소드에 대한 설명 중 틀린 것은 무엇입니까?

① start() 메소드는 실행 대기 상태로 만들어 준다
② sleep() 메소드는 일시 정지 상태로 만들어 준다
③ interrupt() 메소드는 실행 상태를 간섭해서 일시 정지 상태로 만들어 준다 - 정답
④ 일시 정시 상태에서 실행 상태로 변경하는 메소드는 없다

(5) interrupt() 메소드를 호출한 효과에 대한 설명 중 틀린 것은 무엇입니까?

① 일시 정지 상태에서 InterruptedException을 발생시킨다

② 스레드는 즉시 종료한다
③ 실행 대기 상태에서 호출되면 일시 정지 상태가 될 때까지 InterruptedException이 발생하지 않는다
④ 아직 InterruptedException이 발생하지 않았다면 interrupted(), isInterrupted() 메소드는 true를 리턴 한다 –정답

(6) while문으로 반복적인 작업을 하는 스레드를 즉시 종료해야 할 때 안전하게 종료할 수 있는 방법에 대하여 설명하세요

===== 풀이 ====

1. stop 플래그 사용 : 조건식에 boolean 타입의 stop 플래그를 사용하여 반복문을 빠져나가게 한다
2. interrupt() 메소드 사용
   - 스레드가 sleep() 메소드로 일시정지 상태가 될 때 InterruptedException 발생하여 예외 처리 코드에서 break문으로 반복문을 빠져나가게 한다.
   - interrupted() 정적 메소드, isInterrupted() 인스턴스 메소드의 리턴값을 조사하여 true이면 반복문을 빠져나가게 한다.

(7) 메인 스레드에서 1초후에 movie_thread의 interrupt() 메소드를 호출해서 movie_thread를 안전하게 종료하고 싶습니다. 밑줄 친 부분에 적당한 코드를 작성하고 결과를 제시하세요

```
public class ThreadTest {
        public static void main(String[] args) {
                Thread movie_thread = new Thread() {
                        @Override
                        public void run() {
                                int i=0;
                                while(true){
                                        System.out.println(i++ + ") 동영상을 재생합니다.");
                                        if(Thread.interrupted())
                                                break;
                                }
                                System.out.println("자원 정리");
                                System.out.println("실행 종료");
                        }
                };
                movie_thread.start();
                try {
                        Thread.sleep(1000);
                } catch (InterruptedException e) {
                }
                movie_thread.interrupt();    // interrupt() 메소드를 호출하여 movie_thread 스레드 즉시 종료
                System.out.println("메인 스레드 종료");
        }
}
```

[실행 결과]

(8) 메인 스레드가 종료되면 movie_thread도 함께 종료되게 만들고 싶습니다. 밑줄 친 부분에 적당한 코드를 작성하고 결과를 제시하세요

```
public class ThreadTest {
	public static void main(String[] args) {
		Thread movie_thread = new Thread() {
			@Override
			public void run() {
				int i=0;
				while(true){
					System.out.println(i++ + ") 동영상을 재생합니다.");
					try {
						Thread.sleep(1000);
					}
					catch(InterruptedException e) {
						break;
					}
				}
			}
		};
		movie_thread.setDaemon(true);
		movie_thread.start();
		try {
			Thread.sleep(3000);
		} catch (InterruptedException e) {
		}
		System.out.println("메인 스레드 종료");
	}
}
```

[실행 결과]

(9) 제시된 main() 메소드와 결과를 참고하여 다음 설명에 해당하는 프로그램을 작성하세요.

1. Thread를 상속받은 Job 클래스를 작성하세요.

: 필드(private String name, int num)

: 생성자(이름을 전달하여 저장)

: 스레드 메소드 (0.5초간 sleep() 후 getNum()메소드 호출, getNum()에서 리턴 받은 값이 10이면 반복 탈출)

: getNum() (한 순간에 하나의 스레드만 접근할 수 있는 메소드, num값을 1증가 시킨 후 증가된 num값을 리턴)

```
public class Test1 {
```

```
        public static void main(String[] args) {
                Thread t1 = new Job("ThreadA");
                Thread t2 = new Job("ThreadB");
                t1.start();
                t2.start();
        }
}
```

```
ThreadB: 1
ThreadA: 1
ThreadB: 2
ThreadA: 2
ThreadB: 3
ThreadA: 3
ThreadB: 4
ThreadA: 4
ThreadB: 5
ThreadA: 5
ThreadB: 6
ThreadA: 6
ThreadB: 7
ThreadA: 7
ThreadB: 8
ThreadA: 8
ThreadB: 9
ThreadA: 9
```

[프로그램 소스]

```
class Job extends Thread {
        private String name;
        int num = 0;

        public Job(String name) {
                this.name = name;
        }

        public void run() {
                while (true) {
                        try {
                                Thread.sleep(500);
                                int n = getNum();
                                if (n == 10)
                                        break;
                                System.out.println(name + ": " + n);
                        } catch (InterruptedException e) {
                                System.out.println(e.toString());
                        }
                }
        }
        public synchronized int getNum() {
                return ++num;
        }
}

public class Test {
        public static void main(String[] args) {
                Thread t1 = new Job("ThreadA");
                Thread t2 = new Job("ThreadB");
                t1.start();
```

```
                t2.start();
        }
}
```

[실행 결과]

(10) 다음 소스에서 주석의 설명을 고려하여 빈칸을 채우세요.

```
public class ThreadA extends Thread {
        public boolean stop = false;
        public boolean work = true;

        public void run() {
                while(!stop) {
                        //work필드가 true이면 "ThreadA 작업" 출력, false이면 스레드의 실행을 양보한다.
                        if(work==true) System.out.printn("ThreadA")
                        else Thread.yield();
                }
                System.out.println("ThreadA 종료");
        }
}

public class ThreadB extends Thread {
        public boolean stop = false;
        public boolean work = true;

        public void run() {
                while(!stop) {
                        //work필드가 true이면 "ThreadB 작업" 출력, false이면 스레드의 실행을 양보한다.
                        if(work==true) System.out.printn("ThreadA")
                        else Thread.yield();
                }
                System.out.println("ThreadB 종료");
        }
}

public class Example {
        public static void main(String[] args) {
                ThreadA threadA = new ThreadA();
                ThreadB threadB = new ThreadB();
                threadA.start();
                threadB.start();

                try { Thread.sleep(3000); } catch (InterruptedException e) {}
```

```
            threadA.work = false;

            try { Thread.sleep(3000); } catch (InterruptedException e) {}
            threadA.work = true;

            try { Thread.sleep(3000); } catch (InterruptedException e) {}
            threadA.stop = true;
            threadB.stop = true;
        }
}
```